

```
In [14]: !pip install yfinance plotly scikit-learn matplotlib
```

```
Requirement already satisfied: yfinance in E:\PYTHON\Lib\site-  
packages (1.4.1)  
Requirement already satisfied: plotly in E:\PYTHON\Lib\site-  
packages (6.8.0)  
Requirement already satisfied: scikit-learn in E:\PYTHON\Lib\site-  
packages (1.8.0)  
Requirement already satisfied: matplotlib in E:\PYTHON\Lib\site-  
packages (3.10.8)  
Requirement already satisfied: pandas>=1.3.0 in  
E:\PYTHON\Lib\site-packages (from yfinance) (3.0.2)  
Requirement already satisfied: numpy>=1.16.5 in  
E:\PYTHON\Lib\site-packages (from yfinance) (2.4.4)  
Requirement already satisfied: requests>=2.31 in  
E:\PYTHON\Lib\site-packages (from yfinance) (2.33.1)  
Requirement already satisfied: multitasking>=0.0.7 in  
E:\PYTHON\Lib\site-packages (from yfinance) (0.0.13)  
Requirement already satisfied: platformdirs>=2.0.0 in  
E:\PYTHON\Lib\site-packages (from yfinance) (4.9.6)  
Requirement already satisfied: pytz>=2022.5 in E:\PYTHON\Lib\site-  
packages (from yfinance) (2026.2)  
Requirement already satisfied: peewee>=3.16.2 in  
E:\PYTHON\Lib\site-packages (from yfinance) (4.1.0)  
Requirement already satisfied: beautifulsoup4>=4.11.1 in  
E:\PYTHON\Lib\site-packages (from yfinance) (4.14.3)  
Requirement already satisfied: curl_cffi>=0.15 in  
E:\PYTHON\Lib\site-packages (from yfinance) (0.15.0)  
Requirement already satisfied: protobuf>=3.19.0 in  
E:\PYTHON\Lib\site-packages (from yfinance) (7.35.1)  
Requirement already satisfied: websockets>=13.0 in  
E:\PYTHON\Lib\site-packages (from yfinance) (16.0)  
Requirement already satisfied: narwhals>=1.15.1 in  
E:\PYTHON\Lib\site-packages (from plotly) (2.22.1)  
Requirement already satisfied: packaging in E:\PYTHON\Lib\site-  
packages (from plotly) (26.0)  
Requirement already satisfied: scipy>=1.10.0 in  
E:\PYTHON\Lib\site-packages (from scikit-learn) (1.17.1)  
Requirement already satisfied: joblib>=1.3.0 in  
E:\PYTHON\Lib\site-packages (from scikit-learn) (1.5.3)  
Requirement already satisfied: threadpoolctl>=3.2.0 in  
E:\PYTHON\Lib\site-packages (from scikit-learn) (3.6.0)  
Requirement already satisfied: contourpy>=1.0.1 in  
E:\PYTHON\Lib\site-packages (from matplotlib) (1.3.3)  
Requirement already satisfied: cycler>=0.10 in E:\PYTHON\Lib\site-  
packages (from matplotlib) (0.12.1)  
Requirement already satisfied: fonttools>=4.22.0 in  
E:\PYTHON\Lib\site-packages (from matplotlib) (4.62.1)  
Requirement already satisfied: kiwisolver>=1.3.1 in  
E:\PYTHON\Lib\site-packages (from matplotlib) (1.5.0)  
Requirement already satisfied: pillow>=8 in E:\PYTHON\Lib\site-  
packages (from matplotlib) (12.2.0)  
Requirement already satisfied: pyparsing>=3 in E:\PYTHON\Lib\site-
```

```

packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in
E:\PYTHON\Lib\site-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: soupsieve>=1.6.1 in
E:\PYTHON\Lib\site-packages (from beautifulsoup4>=4.11.1-
>yfinance) (2.8.3)
Requirement already satisfied: typing-extensions>=4.0.0 in
E:\PYTHON\Lib\site-packages (from beautifulsoup4>=4.11.1-
>yfinance) (4.15.0)
Requirement already satisfied: cffi>=2.0.0 in E:\PYTHON\Lib\site-
packages (from curl_cffi>=0.15->yfinance) (2.0.0)
Requirement already satisfied: certifi>=2024.2.2 in
E:\PYTHON\Lib\site-packages (from curl_cffi>=0.15->yfinance)
(2026.2.25)
Requirement already satisfied: rich in E:\PYTHON\Lib\site-packages
(from curl_cffi>=0.15->yfinance) (15.0.0)
Requirement already satisfied: pycparser in E:\PYTHON\Lib\site-
packages (from cffi>=2.0.0->curl_cffi>=0.15->yfinance) (3.0)
Requirement already satisfied: tzdata in E:\PYTHON\Lib\site-
packages (from pandas>=1.3.0->yfinance) (2026.1)
Requirement already satisfied: six>=1.5 in E:\PYTHON\Lib\site-
packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
Requirement already satisfied: charset_normalizer<4,>=2 in
E:\PYTHON\Lib\site-packages (from requests>=2.31->yfinance)
(3.4.7)
Requirement already satisfied: idna<4,>=2.5 in E:\PYTHON\Lib\site-
packages (from requests>=2.31->yfinance) (3.11)
Requirement already satisfied: urllib3<3,>=1.26 in
E:\PYTHON\Lib\site-packages (from requests>=2.31->yfinance)
(2.6.3)
Requirement already satisfied: markdown-it-py>=2.2.0 in
E:\PYTHON\Lib\site-packages (from rich->curl_cffi>=0.15->yfinance)
(4.2.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in
E:\PYTHON\Lib\site-packages (from rich->curl_cffi>=0.15->yfinance)
(2.20.0)
Requirement already satisfied: mdurl~=0.1 in E:\PYTHON\Lib\site-
packages (from markdown-it-py>=2.2.0->rich->curl_cffi>=0.15-
>yfinance) (0.1.2)

```

```

In [17]: %matplotlib inline
import matplotlib.pyplot as plt

```

```
In [18]: import yfinance as yf
import pandas as pd
import plotly.graph_objects as go

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import confusion_matrix, accuracy_score,
classification_report

# Input
ticker = input("Enter Stock Ticker (Example: AAPL): ")

# Download stock data
df = yf.download(ticker, period='2y', auto_adjust=False)

# Fix for latest yfinance versions
if isinstance(df.columns, pd.MultiIndex):
    df.columns = df.columns.get_level_values(0)

# Reset index
df = df.reset_index()

# Indicators
df['SMA20'] = df['Close'].rolling(20).mean()
df['EMA20'] = df['Close'].ewm(span=20).mean()

# Daily return
df['Return'] = df['Close'].pct_change()

# Target variable
df['Target'] = (df['Close'].shift(-1) > df['Close']).astype(int)

# Remove missing values
df = df.dropna()

# Features
X = df[['Open', 'High', 'Low', 'Close', 'Volume']]
y = df['Target']

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

# Random Forest Model
model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

model.fit(X_train, y_train)

# Prediction
pred = model.predict(X_test)
```

```
# Accuracy
print("\n===== MODEL PERFORMANCE =====")
print("Accuracy:",
      round(accuracy_score(y_test, pred)*100,2), "%")

# Confusion Matrix
cm = confusion_matrix(y_test, pred)

print("\nConfusion Matrix")
print(cm)

# Classification Report
print("\nClassification Report")
print(classification_report(y_test, pred))

# Confusion Matrix Plot
plt.figure(figsize=(5,4))
plt.imshow(cm)
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.colorbar()
plt.show()

# Feature Importance
importance = pd.DataFrame({
    'Feature': X.columns,
    'Importance': model.feature_importances_
})

print("\nFeature Importance")
print(importance)

plt.figure(figsize=(6,4))
plt.bar(importance['Feature'],
        importance['Importance'])
plt.title("Feature Importance")
plt.show()

# Latest Prediction
latest = model.predict(X.tail(1))[0]

print("\n===== NEXT DAY PREDICTION =====")

if latest == 1:
    print("BUY SIGNAL ↑")
else:
    print("SELL SIGNAL ↓")

# Performance Summary
print("\n===== PERFORMANCE SUMMARY =====")

print("Current Price:",
      round(df['Close'].iloc[-1],2))

print("Highest Price:",
      round(df['High'].max(),2))
```

```
print("Lowest Price:",
      round(df['Low'].min(),2))

print("Average Daily Return:",
      round(df['Return'].mean()*100,2),
      "%")

# Save Dataset
filename = ticker + "_dataset.csv"

df.to_csv(filename, index=False)

print("\nDataset saved as:", filename)

# Price + SMA + EMA Plot
fig = go.Figure()

fig.add_trace(go.Scatter(
    x=df['Date'],
    y=df['Close'],
    name='Close Price'
))

fig.add_trace(go.Scatter(
    x=df['Date'],
    y=df['SMA20'],
    name='SMA20'
))

fig.add_trace(go.Scatter(
    x=df['Date'],
    y=df['EMA20'],
    name='EMA20'
))

fig.update_layout(
    title=ticker + " Stock Analysis",
    xaxis_title="Date",
    yaxis_title="Price"
)

fig.show()

# Candlestick Chart
fig2 = go.Figure()

fig2.add_trace(
    go.Candlestick(
        x=df['Date'],
        open=df['Open'],
        high=df['High'],
        low=df['Low'],
        close=df['Close']
    )
)
```

```
fig2.update_layout(  
    title=ticker + " Candlestick Chart"  
)  
  
fig2.show()
```

Enter Stock Ticker (Example: AAPL): `tsla`

[*****100%*****] 1 of 1
completed

===== MODEL PERFORMANCE =====

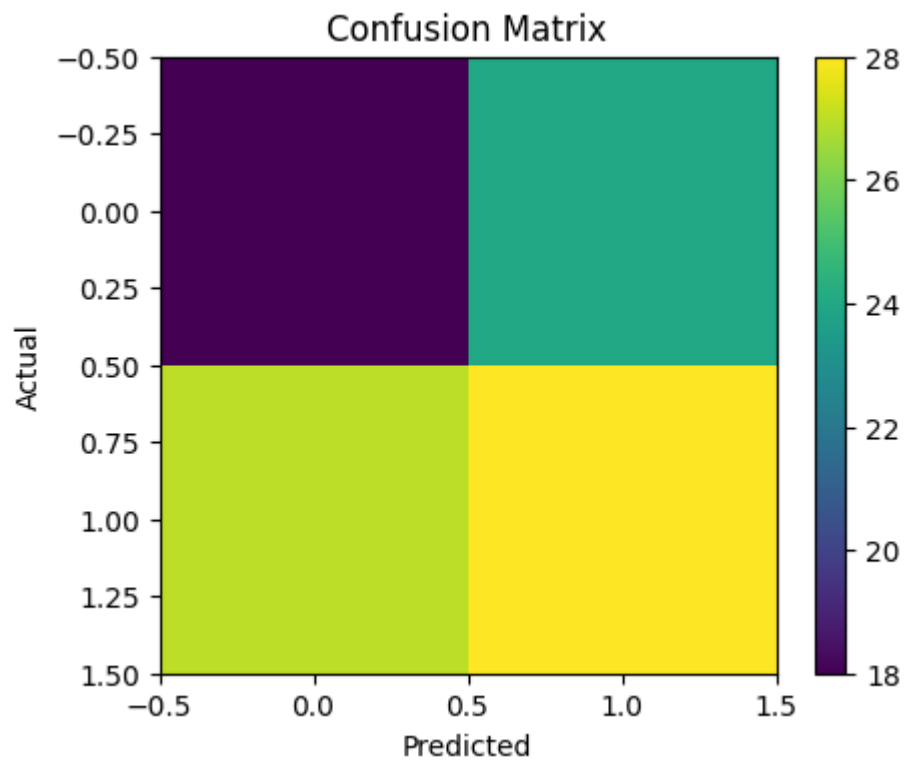
Accuracy: 47.42 %

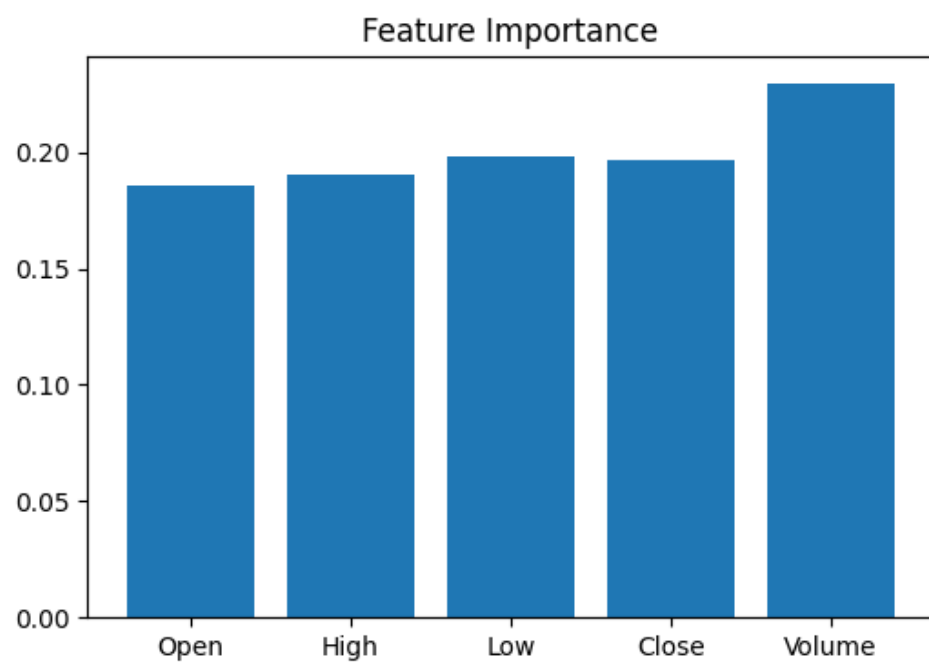
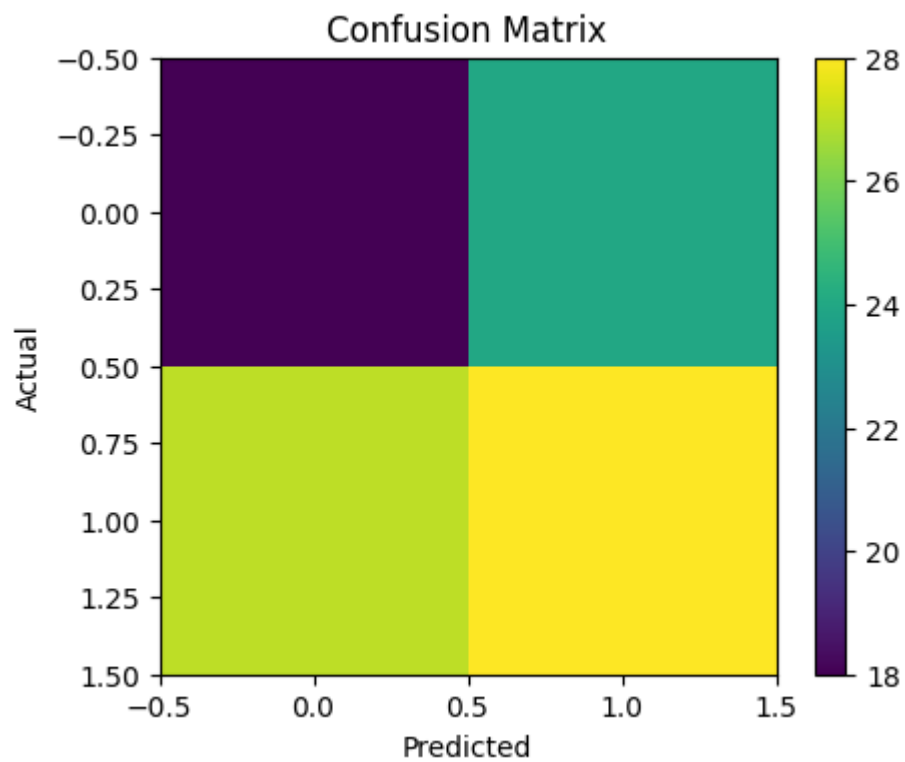
Confusion Matrix

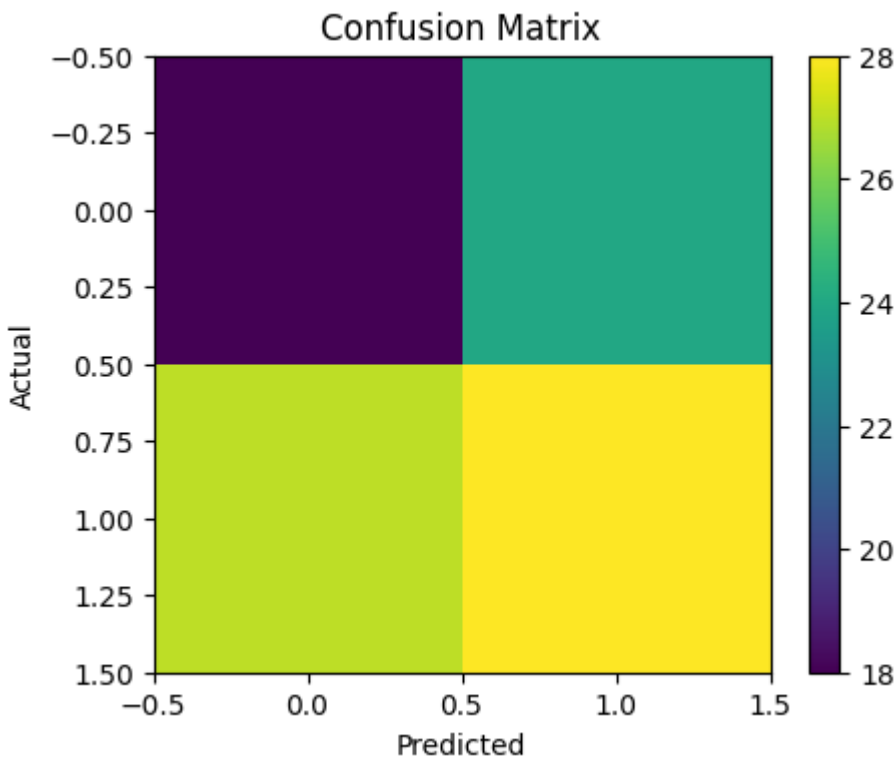
```
[[18 24]  
 [27 28]]
```

Classification Report

	precision	recall	f1-score	support
0	0.40	0.43	0.41	42
1	0.54	0.51	0.52	55
accuracy			0.47	97
macro avg	0.47	0.47	0.47	97
weighted avg	0.48	0.47	0.48	97

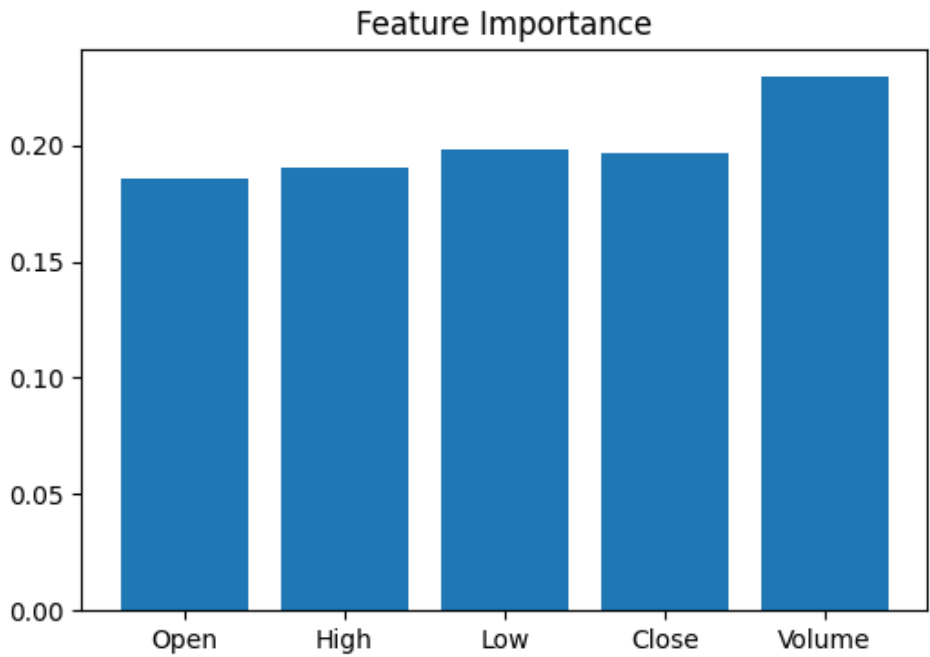






Feature Importance

	Feature	Importance
0	Open	0.185714
1	High	0.190586
2	Low	0.197868
3	Close	0.196386
4	Volume	0.229446



===== NEXT DAY PREDICTION =====
SELL SIGNAL ↓

===== PERFORMANCE SUMMARY =====

Current Price: 375.12
Highest Price: 498.83
Lowest Price: 182.0
Average Daily Return: 0.16 %

Dataset saved as: tsla_dataset.csv

In []:

Exported with [runcell](#) — convert notebooks to HTML or PDF anytime at [runcell.dev](#).